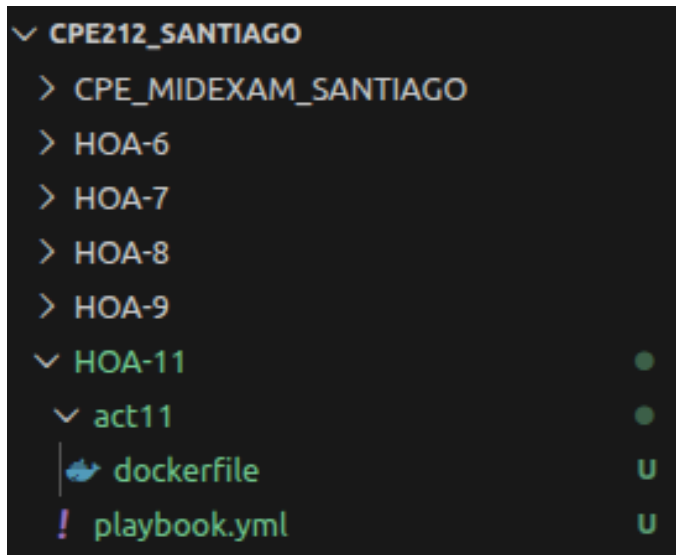


Name: Santiago, Jalene	Date Performed: March 19, 2026
Course/Section: CPE212 - CPE31S1	Date Submitted: March 19, 2026
Instructor: Engr. Robin E. Valenzuela	Semester and SY: 2nd Sem - 25-26
Activity 11: Containerization	
1. Objectives	
Create a Dockerfile and form a workflow using Ansible as Infrastructure as Code (IaC) to enable Continuous Delivery process	
2. Discussion	
Docker is an open platform for developing, shipping, and running applications. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly. With Docker, you can manage your infrastructure in the same ways you manage your applications. By taking advantage of Docker's methodologies for shipping, testing, and deploying code quickly, you can significantly reduce the delay between writing code and running it in production. Source: https://docs.docker.com/get-started/overview/ You may also check the difference between containers and virtual machines. Click the link given below. Source: https://docs.microsoft.com/en-us/virtualization/windowscontainers/about/containers-vs-vm	
3. Tasks	
1. Create a new repository for this activity. 2. Install Docker and enable the docker socket. 3. Add to Docker group to your current user. 4. Create a Dockerfile to install web and DB server. 5. Install and build the Dockerfile using Ansible. 6. Add, commit and push it to your repository.	
4. Output (screenshots and explanations)	
Create a new repository for this activity. 	

Install Docker and enable the docker socket.

```
jalene@workstation:~$ docker --version
Docker version 29.3.0, build 5927d80
jalene@workstation:~$
```

```
jalene@workstation:~$ sudo systemctl start docker
jalene@workstation:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2026-03-19 22:02:34 +08; 32s ago
 TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 4918 (dockerd)
       Tasks: 9
      Memory: 41.2M
         CPU: 703ms
    CGroup: /system.slice/docker.service
            └─4918 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
```

This ensures the system can run containers and allows Docker to operate properly in the background.

Add to Docker group to your current user.

```
jalene@workstation:~$ sudo usermod -aG docker $USER
jalene@workstation:~$ newgrp docker
jalene@workstation:~$
```

This allows the user to execute Docker commands without needing administrative privileges.

Create a Dockerfile to install web and DB server.

```
! playbook.yml u  dockerfile u x
HOA-11 > dockerfile
1  FROM ubuntu:22.04
2
3  ENV DEBIAN_FRONTEND=noninteractive
4
5  RUN apt-get update && \
6      apt-get install -y apache2 mysql-server && \
7      apt-get clean
8
9  EXPOSE 80 3306
10
11 CMD service mysql start && apachectl -D FOREGROUND
```

The Dockerfile creates a self-contained environment using the ubuntu:22.04 base image, installing both Apache as the web server and MySQL as the database server. By setting DEBIAN_FRONTEND=noninteractive, it prevents interactive prompts during installation. The Dockerfile exposes ports 80 and 3306 for external access and uses a single CMD instruction to start both services simultaneously, eliminating the need for an external start script.

```

! playbook.yml U  dockerfile U
HOA-11 > ! playbook.yml
1 ---
2 - name: Build and Run Docker Container
3   hosts: localhost
4   become: yes
5   tasks:
6
7     - name: Ensure docker Python module is installed
8       pip:
9         name: docker
10        state: present
11
12    - name: Build Docker image from Dockerfile
13      community.docker.docker_image:
14        name: web-db-server
15        source: build
16        build:
17          path: .
18          tag: latest
19          state: present
20
21    - name: Ensure the container is running
22      community.docker.docker_container:
23        name: mycontainer
24        image: web-db-server:latest
25        state: started
26        recreate: yes
27        ports:
28          - "8080:80"
29          - "3306:3306"

```

The Ansible playbook complements the Dockerfile by automating the entire workflow. It ensures Docker is installed on the host and that the Python docker module is available. The playbook builds the Docker image from the Dockerfile and automatically recreates the container if there are updates, mapping the necessary ports for Apache and MySQL. This setup eliminates manual Docker commands, providing a reproducible, fully automated deployment process. Together, the Dockerfile and playbook create a self-sufficient system for running a web server and database in a single container.

Install and build the Dockerfile using Ansible.

```

● jalene@workstation:~/CPE212_Santiago/HOA-11$ ansible-playbook playbook.yml -K
BECOME password:
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost
does not match 'all'

PLAY [Build and Run Docker Container] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Ensure docker Python module is installed] *****
ok: [localhost]

TASK [Build Docker image from Dockerfile] *****
changed: [localhost]

TASK [Ensure the container is running] *****
[DEPRECATION WARNING]: The container_default behavior option will change its default value from
"compatibility" to "no_defaults" in community.docker 2.0.0. To remove this warning, please specify an
explicit value for it now. This feature will be removed from community.docker in version 2.0.0.
Deprecation warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.
changed: [localhost]

PLAY RECAP *****
localhost                : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

```

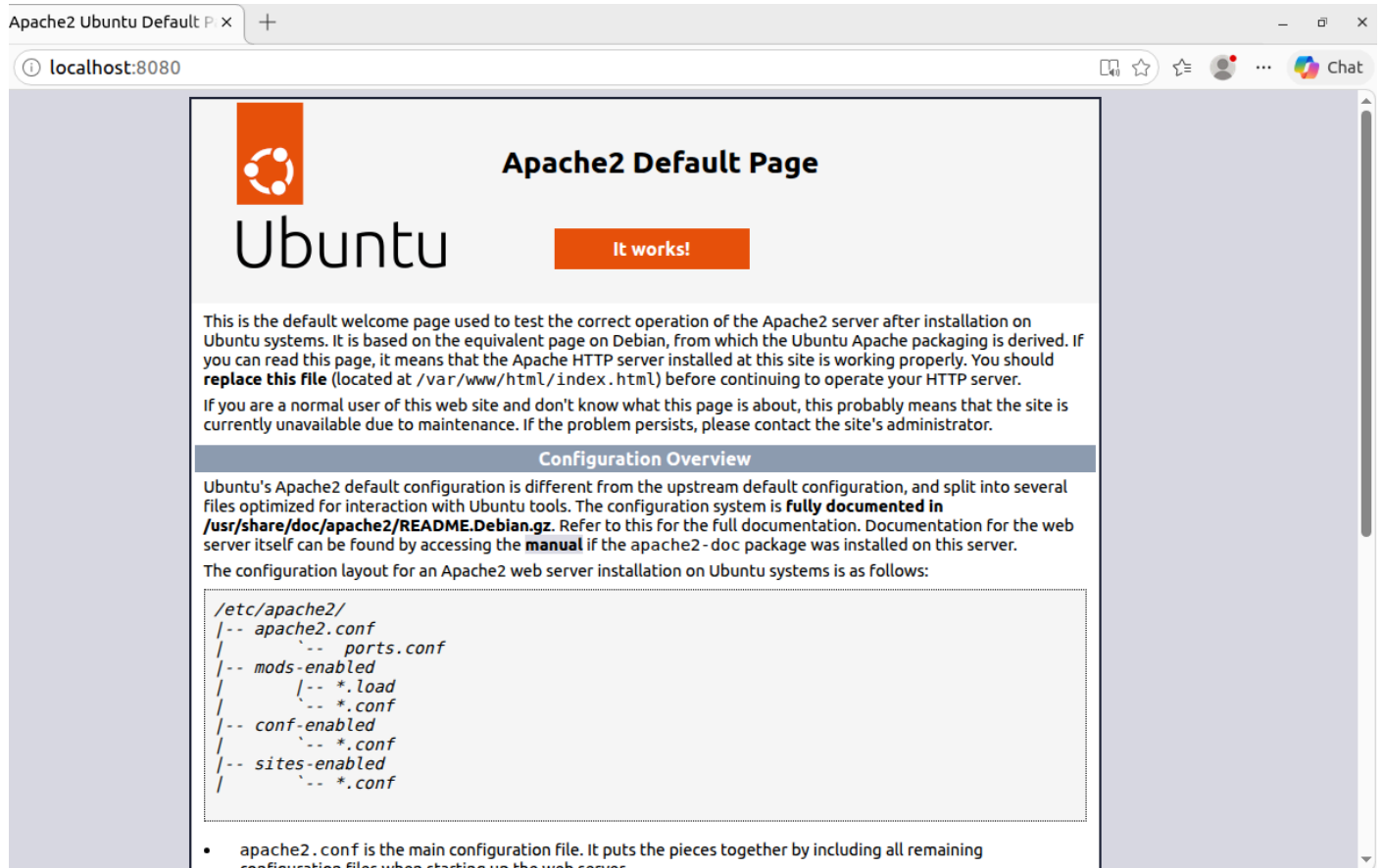
```

jalene@workstation:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED        STATUS        PORTS
69e42e6c45ad   web-db-server:latest               "/bin/sh -c 'service..."           37 seconds ago Up 36 seconds 0.0.0.0:3306->3306/tcp, 0.0.0.0:8080->80/tcp
jalene@workstation:~$

```

This automates the process of building the Docker image.

VERIFICATION



```

jalene@workstation:~$ docker exec -it mycontainer mysql -uroot
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.45-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>

```

Add, commit and push it to your repository.

```
jalene@workstation:~/CPE212_Santiago$ git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 3 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 876 bytes | 876.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
```



jalene-s Docker + Ansible (Web and Database)

4bc235b · 2 minutes ago



10 Commits



HOA-11

Docker + Ansible (Web and Database)

2 minutes ago

Reflections:

Answer the following:

1. What are the benefits of implementing containerizations?

Containerization provides a consistent environment where applications can run regardless of the system. It improves deployment speed since all dependencies are already packaged within the container. Compared to virtual machines, containers are lightweight and use fewer resources. It also makes scaling and managing applications easier, especially in cloud environments.

Conclusions:

This activity focuses on creating a Dockerfile and using Ansible to automate the build process. I was able to set up a containerized environment with both a web server and a database server. The use of Ansible helped simplify and automate the workflow, making deployment more efficient.